

Tomahawk 17315



Software Portfolio

2025-2026





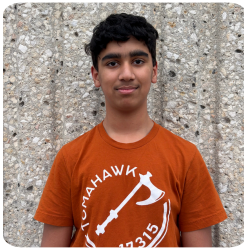
Our Software Team



Aditya Rayapeddi
Software Lead

Junior at Westwood High School

This is my third year in FTC, and I have more than 5 years of Java experience, with certifications showing my exemplary knowledge. I first got into robotics in middle school, as part of the Vex Robotics Competition. I also now intern at a private robotics company, where I program a robot dog and its arm. In my free time, I love watching and playing sports like soccer and basketball, and playing video games.



Minessh Uchat
Autonomous

Freshman at Westwood High School

I have done one year in VEX robotics, and I was the software lead for our team. I learned a lot about teamwork, leadership, and also met many new people who could help me in the future. At home, I continue my work as I make small projects that I both enjoy and can learn many useful skills from, including electronics, programming, and digital logic.



Sanaika Nandigam
Documentation

Freshman at Westwood High School

I was the captain of a Vex VRC Robotics team for one year where I learned strong skills like leadership, teamwork, and problem-solving. I progressed into a VEX V5 robotics team in which I was there for about 3-4 months in the software team. In Vex V5 I learned most of my prominent robotics skills and dwelled into the realm of robotics. FTC has always been a goal I wanted to achieve, so this year is almost like a dream for me! In my free time, I like to play flute, sing, and read.

Mentors



Demian Medve
Former FTC Teams 19746,
20716 Software Lead



Josh Eversmann
Adult mentor



Nipun Nagendra
6210 Alumni, Software &
Awards mentor



Subsystems - Drivetrain

KEY COMPONENTS

The DriveTrain class is a subsystem for an FTC (First Tech Challenge) robot that manages the robot's movement. It handles two primary modes of control: **Robot-Centric and Field-Centric**, allowing the robot to be driven relative to its own orientation or the playing field's orientation, respectively.

KEY COMPONENTS

- **Motors:** The class controls four DcMotorEx objects, representing the front-left (fl), front-right (fr), back-left (bl), and back-right (br) wheels.
- **IMU:** It uses an Inertial Measurement Unit (IMU) to **get the robot's heading**, which is crucial for Field-Centric driving.
- **Gamepad:** The gamepad1 **object provides input from the driver's controller, using the left stick for movement and the right stick for turning.**
- **State Enum:** The State enumeration, with values ROBOTCENTRIC and FIELDCENTRIC, **defines the current driving mode**

DRIVING LOGIC

The core functionality lies in the update() method, which **reads inputs from the gamepad** and translates them into **motor power values**. It calculates and applies the correct **power for each motor based on the robot's current state**. Below is a diagram displaying the individual wheel rotations necessary to move the robot in a specific direction in the mecanum drivetrain system.

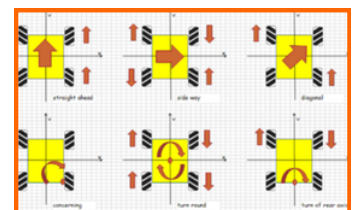
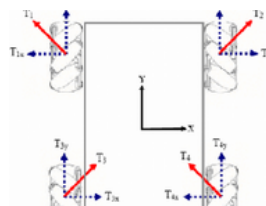
ROBOT-CENTRIC DRIVING

In this mode, the **robot's movement is relative to its current orientation**

- The y input controls forward/backward movement.
- The x input controls strafing (side-to-side) movement.
- The rx input controls rotation.
- The powers are calculated using a simple formula that sums these inputs. **A denominator is used to scale all powers so they don't exceed 1.0**, preventing motors from being commanded to go beyond their maximum speed.

The formulas used are:

- Power for front left wheel = $(y + x + rx) / \text{denominator}$
- $\text{frPower} = (y - x - rx) / \text{denominator}$
- $\text{blPower} = (y - x + rx) / \text{denominator}$
- $\text{brPower} = (y + x - rx) / \text{denominator}$



Shows the motor/axis direction based on



Subsystems - Outtake

Odometry

We use Pinpoint odometry to track the robot's position. From that, the robot can tell:

- where it is
- where the goal is
- how far away the goal is

That distance is the main thing the robot needs to calculate the shot.

RPM

Once the robot knows the distance, it picks the right RPM for the flywheel.

- close = lower RPM
- far = higher RPM

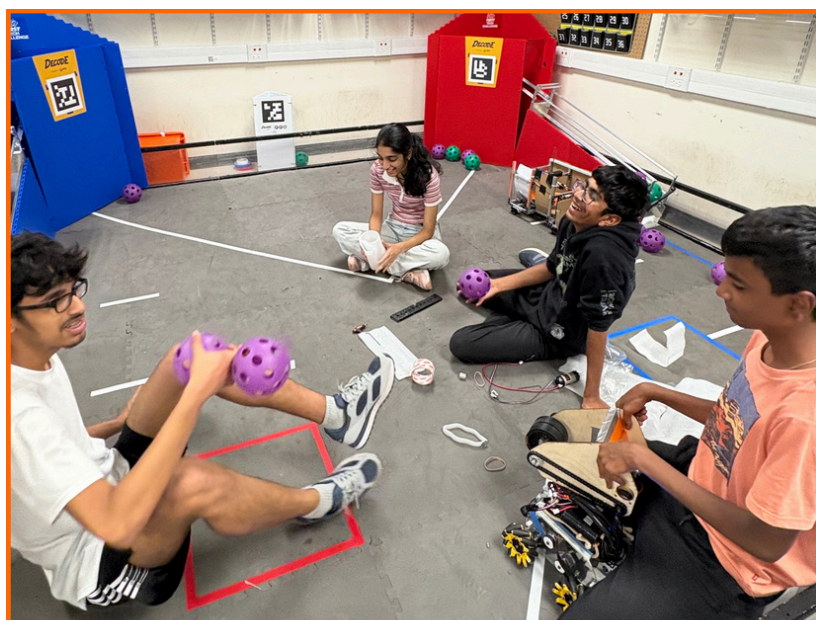
We tested a bunch of values (1500 → 2000 → 3100 RPM) until we found the speeds that consistently reached the goal.

Hood

The hood controls the arc of the shot.

- close shots = lower hood angle
- far shots = higher hood angle

We made a simple chart that maps distance → hood angle, and the robot adjusts automatically.



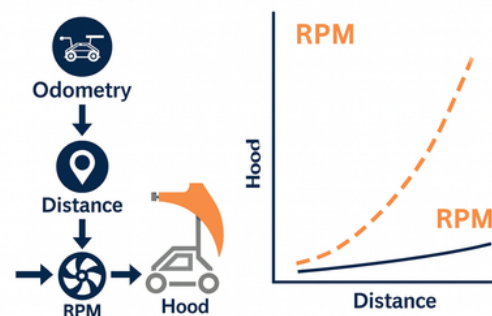
Testing our Outtake Shooter

Process

When the robot wants to shoot:

1. Odometry gives the robot's position
2. Robot calculates distance to the goal
3. Distance sets the flywheel RPM
4. Distance sets the hood angle
5. Turret turns toward the goal
6. Flywheel stabilizes
7. Robot fires

Odometry-based auto-velocity shooting system



Auto-Velocity

Auto-velocity makes our shots:

- more consistent
- faster
- easier for drivers
- reliable from different spots on the field

It's one of the main reasons our outtake works so well.



Subsystems - Flywheel improvements

Throughout the season, we made major upgrades to our flywheel system to improve consistency, recovery time, and overall scoring reliability. These improvements focused on control algorithms, feeding logic, and tuning.

Bang-Bang And PID

At the start of the season, we used a bang-bang controller for the flywheel. Bang-bang is simple:

- if RPM is below target → full power
- if RPM is above target → cut power

It spins up fast, but it caused:

- RPM bouncing
- inconsistent shots
- slow recovery between balls

We replaced it with PID velocity control, which gave us:

- smoother acceleration
- stable RPM
- faster recovery
- consistent multi-shot cycles

This was one of the biggest improvements to our scoring system.

agraph text

Transfer Wheel Logic

We added a transfer wheel between the intake and the flywheel to prevent accidental feeding.

The transfer wheel:

- stops balls from entering the flywheel too early
- only feeds when the flywheel is stable
- prevents double-feeds
- reduces jams

This made our entire scoring pipeline more controlled.

FSM (Finite State Machine) Control

To manage the intake → transfer → outtake flow, we built a simple FSM with states like:

- INTAKING
- READY
- SHOOTING

Each state controls which motors are allowed to run. This prevents the intake from fighting the outtake and keeps timing consistent.

We also added a manual override so drivers can clear jams or reset the system instantly.

Flywheel Tuning

We spent a lot of time tuning:

- PID values
- RPM targets
- recovery time
- distance-based velocity adjustments

We tested RPM ranges like 1500 → 2000 → 3100, and built a distance-to-velocity curve so the robot could shoot accurately from different spots.

These tuning sessions made our flywheel stable enough for high-speed auton cycles.

Bang-Bang vs. PID Velocity Control

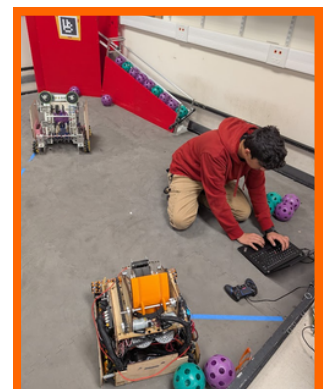
BANG-BANG CONTROL → **PID VELOCITY CONTROL**

- If RPM < Target RPM → full power
- If RPM > Target RPM → cut power

- Smooth acceleration
- Stable RPM
- Faster recovery
- Consistent shots



— Flywheel RPM — Target RPM





Subsystems - Turret

At the beginning of the season, we developed a turret that would use a motor and a belt-pulley system; however, after careful consideration, we switched to a system using two Axon servos in Continuous Rotation (CR) mode.

PROBLEM: We hit a dead end with our use of a public library made to interact with an Axon and an encoder in conjunction to get the current position of the servo. The Axon did not move in accordance with the commands provided to it.

SOLUTION: We implemented some of the same logic that the external library used into our own Turret class, where we figured out the formula to get the position of the servo from the raw voltage of the encoder, and used that to implement an update function that used the error from the desired position and the actual position to calculate the power to be set to the Axons.



BOUNDS

Due to the nature of the wiring of our turret, we had to create bounds so that the turret's motion did not disrupt the wiring allocations. We used custom encoder logic that allowed rotation in each direction of 180 degrees from the starting, forward-facing position. This is so that we can keep our 360 degree motion while not tripping over the wiring.

OFFSETS

Due to minor slips in the gearing of the Axon to the turret, the turret had a different position it assumed to be zero for each time we ran the servo. Our solution for this was to create an initializer class that stored the first recorded value of the turret's rotation as the initial angle, and the zero angle was reset to the initial angle to account for the slips without regearing each time.

AUTO-ALIGN

To make shooting the balls easier with the use of the turret, we implemented an automatic alignment system, or auto-align for short, so that whenever the driver decided to shoot (turning on the flywheel and the transfer to let the ball through), we could align the turret so that no matter the rotation of the bot the flywheel would always shoot in the direction of the goal.

ENCODERS

To accurately get the real-time rotation of the turret, we used analog encoders connected to the Axon servos using a custom board and used the voltage value returned by these encoders to calculate the angle in degrees that the turret pointed towards from the starting position. The formula for this (not including offsets) is:

$$\theta = (\text{encoder voltage value} / 3.3 * 360) \% 360$$

AUTON

To be able to fit our 18-ball autonomous program within 30 seconds (the autonomous period), we had to cut down time. We recognized that one of the key drains of time was in turning to face the goal to shoot. We eliminated this with the use of the turret in each of the sequences which required shooting, where in any angle we were able to shoot, drastically reducing cycle times.



Subsystems - Intake

Our intake subsystem is designed to collect artifacts quickly without accidentally feeding them into the outtake at the wrong time. To keep everything controlled and consistent, we use a transfer wheel and a finite state machine (FSM) that manages when each part of the system is allowed to run.

HOW THE INTAKE SYSTEM ACTUALLY WORKS

The intake has two main parts:

- Front intake wheels : pull the artifact into the robot
- Transfer wheel : moves the artifact from the intake to the outtake

The transfer wheel is the key piece that prevents jams and accidental firing.

WHY WE USE A TRANSFER WHEEL

Without a transfer wheel, artifacts could:

- roll straight into the flywheel
- get stuck halfway
- cause double-feeds
- fire before the flywheel is ready

The transfer wheel acts like a gatekeeper. It only moves the artifact forward when the robot is actually ready to shoot.

FINE STATE MACHINE(FSM) CONTROL

To keep everything organized, we use a state-based system. The robot is always in one of these states:

1. INTAKING

- intake wheels ON
- transfer wheel OFF
- outtake blocked

This prevents the artifact from entering the flywheel too early.

2. TRANSFERRING / READY

- transfer wheel moves the artifact into position
- flywheel spins up
- turret aligns

3. OUTTAKING (Shooting)

- transfer wheel feeds the artifact
- flywheel fires
- intake temporarily disabled

This ensures only one artifact enters the flywheel at a time.

WHY FSM MATTERS

The FSM prevents:

- accidental double-feeds
- artifacts slipping into the flywheel
- Jams during auton
- misfires during TeleOp
- the intake from fighting the outtake
- It keeps the entire scoring system predictable

MANUAL OVERRIDE

Even with a good FSM, things can go wrong:

sensors misread
artifacts bounce weirdly
timing gets off
the robot gets hit

To handle this, we added a manual override button that lets the driver:

- force the transfer wheel
- reverse the intake
- clear jams
- reset the FSM state

This makes the system reliable even under pressure.



Photo of our latest version

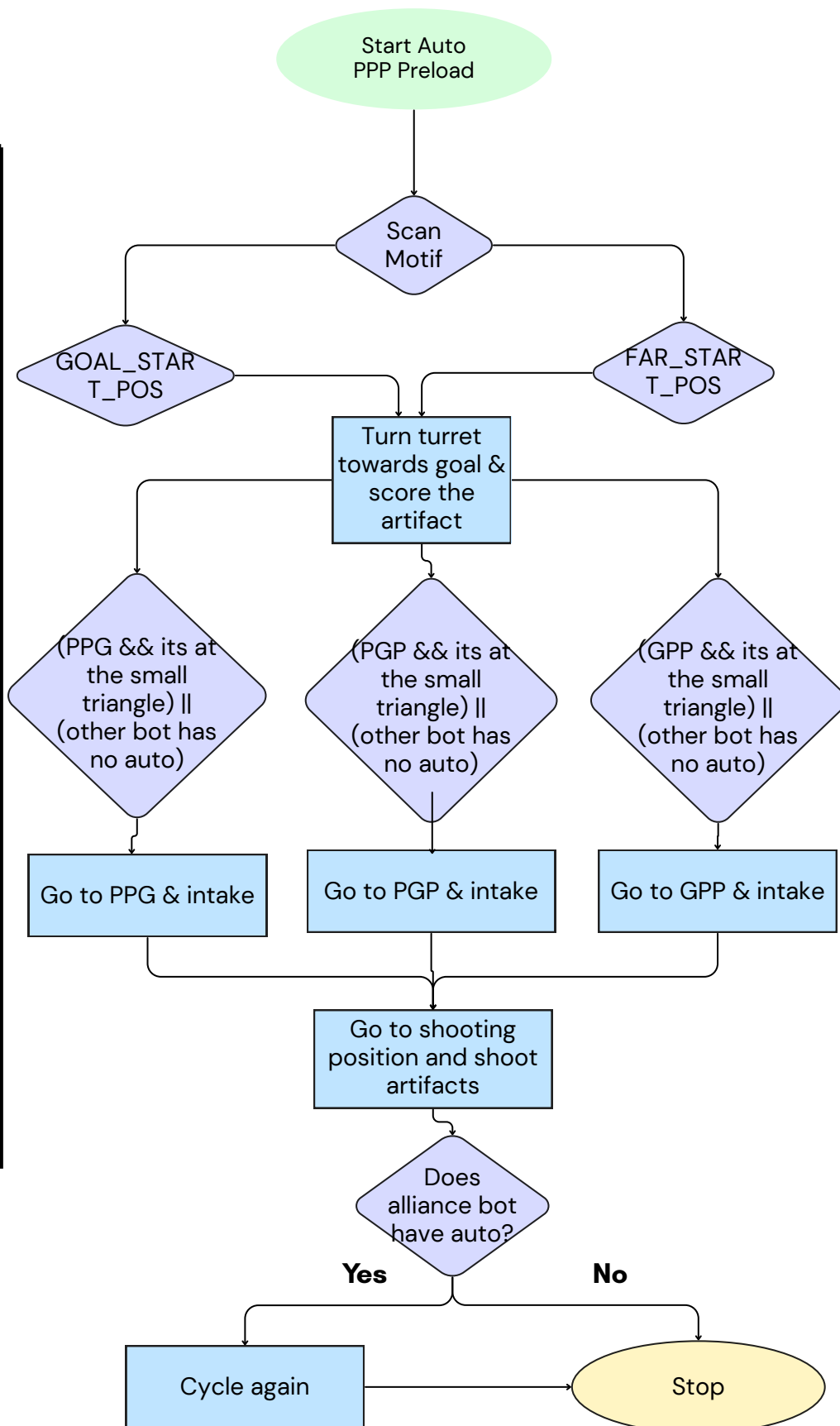


Auto Pathing Flowchart

KEY COMPONENTS

AUTO LOGIC FOR DIFFERENT MOTIFS

This flowchart outlines our autonomous pathing strategy for handling different motifs. The process begins by identifying the start point and scanning the assigned motif. From there, two possible starting conditions are evaluated: goal start pose and far start pose. Regardless of the starting condition, both paths converge on a unified solution: turning the turret toward the goal and scoring the artifact. The specific outcomes and pathing adjustments vary depending on the three distinct motif types. Each motif introduces unique routing considerations and scoring behaviors, which will be detailed in the following sections.





AutoPath Strategy & Optimization

Main Idea

Our autonomous routine wasn't just about following a path, it was about making smart decisions on when to move, what to pick up first, and how to avoid collisions with partner teams. This page explains the strategic choices behind our pathing and how we optimized the routine for speed, safety, and flexibility.

Opening the Gate Efficiently

The first challenge in auton is opening the gate without wasting time. We optimized this step by:

- approaching the gate at a consistent angle so the robot doesn't bounce or get misaligned
- timing the gate push so it opens fully without slowing the robot down
- starting intake spin-up early so the robot is ready to collect immediately after the gate opens

This reduced the "dead time" at the start of auton and made the rest of the cycle smoother.

Choosing the Order of Artifacts

The order in which we collect artifacts has a huge impact on cycle speed. We tested multiple sequences and settled on the one that gave us:

- the shortest total driving distance
- the cleanest intake angle
- the least turret rotation
- the most consistent shot timing

By prioritizing the artifact closest to our first shooting position, we reduced unnecessary turns and made the first cycle significantly faster.

Avoiding Collisions With Partner Teams

A major part of auton strategy is making sure we don't interfere with our alliance partner. To stay partner-safe, we:

- shifted our starting angle so our first movement never crosses their path
- adjusted our intake direction to avoid sweeping into their lane
- added alternate routes in case our partner runs a conflicting auton
- kept our first cycle tight so we clear the shared space quickly

This made our auton reliable even when paired with teams running aggressive or unpredictable routines.

Flexibility

We built several versions of our auton so we could adapt to different match situations:

- full 3-artifact cycle
- safe 2-artifact cycle when partners run long paths
- short cycle for matches where we prioritize TeleOp setup
- mirrored versions for red and blue sides

This flexibility meant we always had a path that fit the alliance strategy.

Why We Split the Auton Into Multiple Actions

Originally, our entire auton was one long action. We split it into smaller actions because:

- sequential actions let us control timing more precisely
- parallel actions let us intake while driving or spin up the flywheel while turning
- modular actions made debugging easier
- mix-and-match actions let us build multiple auton variants quickly

This structure is what allowed us to optimize the routine without rewriting everything from scratch.

These auto paths are for blue side. The red side path is the same thing but mirrored.



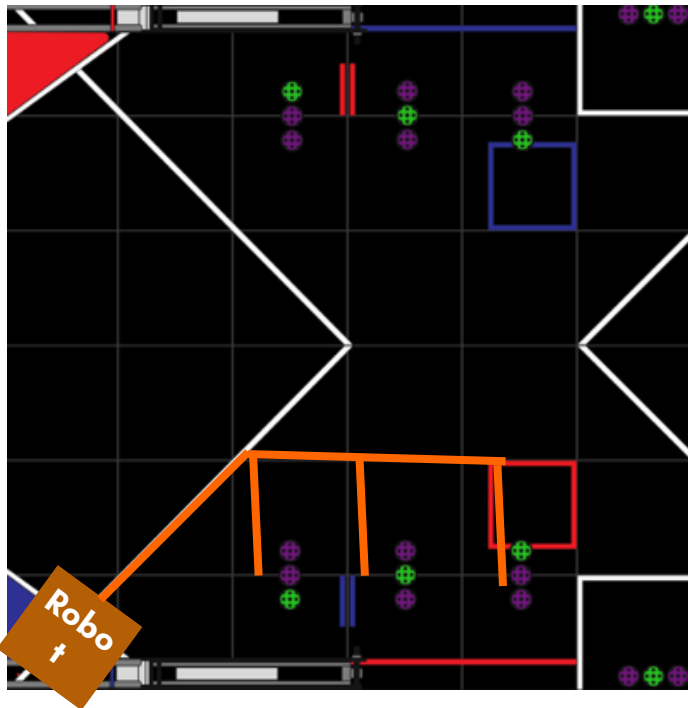
LM2 Autonomous

MAJOR PROBLEM/MAIN TOPIC

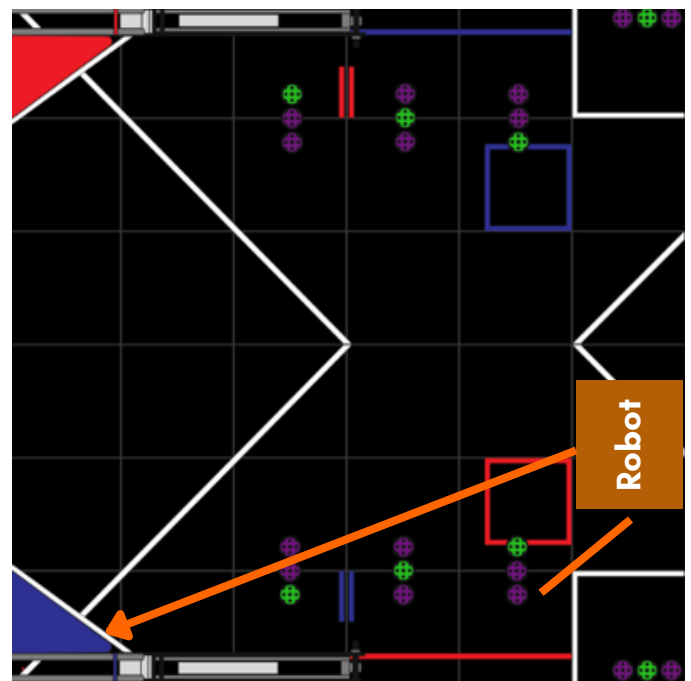
nothing

PATHS/ SOLUTIONS

There was improvement in our autonomous paths in LM2 since we had a close auto of 9 artifacts. In the tournament, however, we were scoring 6-8 balls. Our far auto also had seen improvement. Just like LM1, the change of the field's was presumably the reason for different scoring.



- Shoot
- Intake
- Go back
- Intake the other three artifacts
- Go back
- Intake the other three artifacts



- Intake the three artifacts
- Shoot the three artifacts
- Go back and shoot them
- Repeat

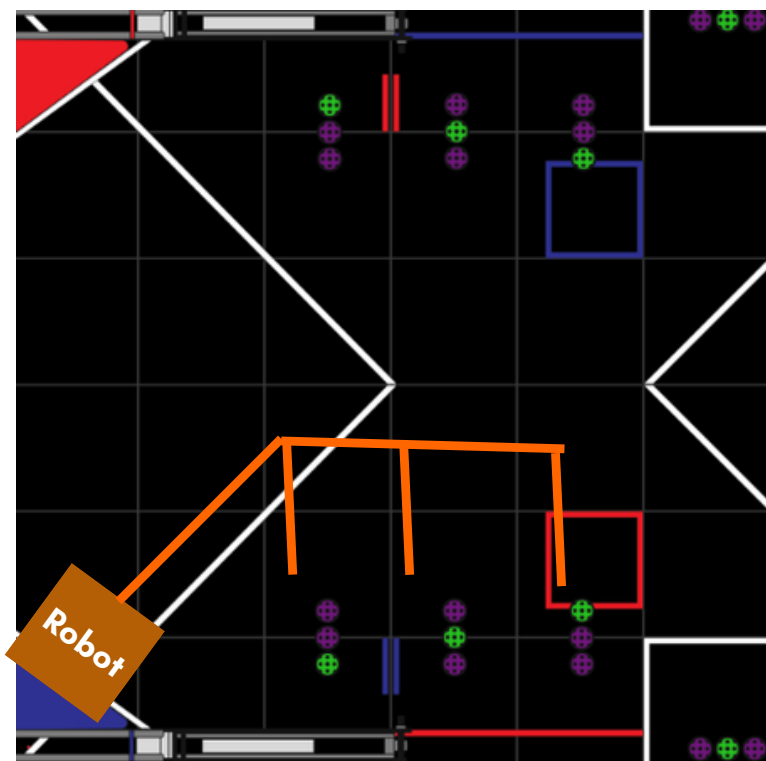
These auto paths are for blue side. The red side path is the same thing but mirrored.



LM3 Autonomous

MAJOR PROBLEM/MAIN TOPIC

This autonomous routine is a streamlined version of our original 12-ball path. Instead of attempting to score all three artifacts from the second cycle, we focused on consistency and time management. After scoring the first two cycles (preload + first two spikes), we drive to the third spike, collect it, and then park at the gate before time runs out.



PATHS/ SOLUTIONS

Our goal for this routine was simple:

- score reliably early
- avoid last-second rushes
- guarantee a clean park
- reduce risk of missed shots or jams

1. Preload Shot

- Robot starts aligned to the goal
- Flywheel spins up
- Turret locks on
- We shoot the preload immediately for a fast first score

This gives us early points and sets the pace for the rest of the routine.

2. Robot drives to the first spike mark

- Intake runs while driving
- Artifact is collected on the move
- We turn back toward the shooting position
- This keeps the cycle fast and avoids wasted time.

4. Robot drives to the second spike

- Intake runs continuously
- Artifact is picked up cleanly
- Robot rotates back toward the goal
- We optimized this path to avoid drifting into partner space.

5. Robot returns to the same shooting position

Flywheel reaches target RPM
Turret aligns
We fire the second spike artifact
At this point, we've scored preload + 2 spike artifacts.

6. Robot returns to the same shooting position

Flywheel reaches target RPM
Turret aligns
We fire the second spike artifact
At this point, we've scored preload + 2 spike artifacts.

7. Robot drives directly to the gate

We park cleanly before the timer ends
This guarantees the park points and avoids any last-second chaos.

3.

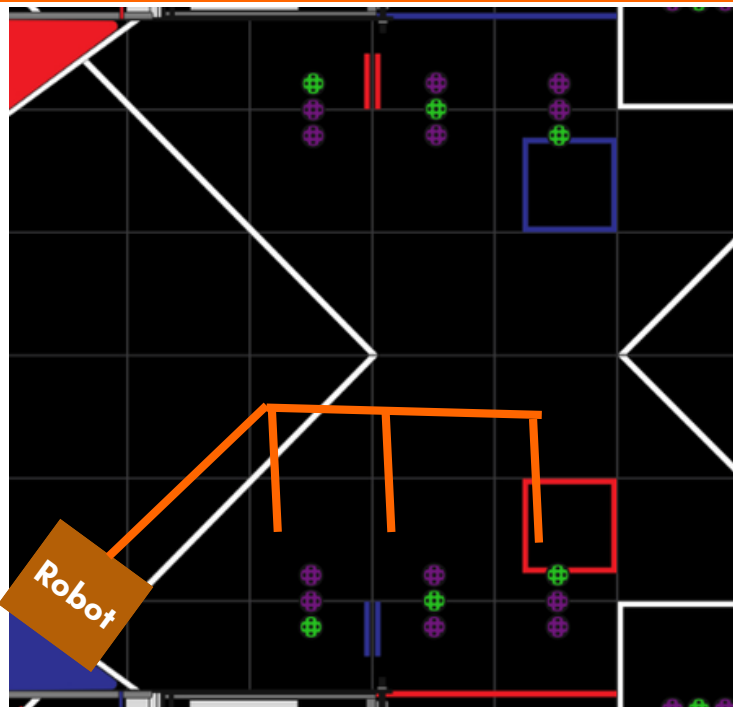
- Robot returns to the shooting spot
- Flywheel stabilizes
- Turret aligns
- We fire the first spike artifact
- This completes the first full cycle.



LC's Autonomous

MAJOR PROBLEM/MAIN TOPIC

This is our highest-scoring and most aggressive autonomous routine. In a perfect scenario, the robot collects all three spike artifacts plus all three gate-drop artifacts, and then sweeps the Human Player zone for the extra balls that fall out when the gate opens. This routine requires precise timing, fast intake, and stable auto-velocity shooting.



6. Final Shots + Park
Any remaining balls are shot. Robot drives to the gate. Parks cleanly before the timer ends. This finishes the routine with maximum points.

5. Sweep the HP Zone
When the gate opens, extra balls fall into the Human Player zone. We drive into that area and: intake all loose balls, keep the flywheel spinning, shoot as we go, maintain turret alignment while moving. This is where the auton jumps from 12 balls to 18 balls in a perfect run.

4. Intake + Score GPP (Gate Drop → Third Spike → Third Spike)

- Drive to the third gate-drop ball
- Intake it
- Shoot it
- Drive to the third spike
- Intake the spike artifact
- Shoot it

 This completes the GPP cycle. At this point, we've scored 6 artifacts and collected 6 balls.

OVERVIEW

Advanced High-Score Auton:

- Score Preload & All Artifacts: Clear all three spike and three gate-drop artifacts.
- Zone Sweep: Clear the HP zone for extra ball scoring.
- Max Efficiency: Rapid-fire scoring until the timer expires.

1. Preload Shot

- Robot starts aligned
- Flywheel spins up
- Turret locks on
- Preload is fired immediately

This gives us instant points and starts the cycle fast.

2. Intake + Score PPG (Preload → First Spike → Gate Drop #1)

- Drive to first spike
- Intake picks up the artifact
- Return to shooting spot
- Shoot the first spike artifact
- Rotate toward the first gate-drop ball
- Intake the gate-drop ball
- Shoot it immediately

This completes the PPG cycle.

3. Intake + Score PGP (Gate Drop → Second Spike → Gate Drop #2)

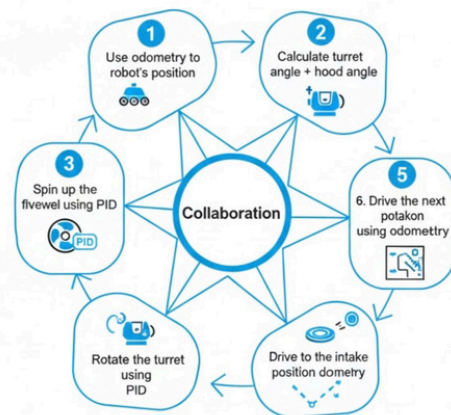
- Drive to the second spike
- Intake the spike artifact
- Shoot it
- Turn toward the second gate-drop ball
- Intake it
- Shoot it
- This completes the PGP cycle.



Collaboration

How These Systems Work Together in Auto

Each cycle of our autonomous routine follows the **same** structure. Because every step is **math-driven** and **PID-stabilized**, the robot behaves the same way every time, even on **different fields**. The odometry gives us a **reliable position**, the geometry tells the **turret** and **hood** exactly where to **aim**, and the PID loops make sure the motors actually **reach** and **hold** the values we **calculated**. This keeps the whole routine **predictable**, so even if the carpet is **worn down** or the field has **slightly different friction**, the robot still drives to the **same spots and shoots** with the same **power**. It basically removes the **"randomness"** from autonomous and makes the robot act the way we **expect** every run.



1 Use **odometry** to determine the robot's **position**

2 Calculate **turret angle** + **hood angle** using **geometry**

3 Spin up the **flywheel** using **PID and bang-bang**

4 **Rotate** the **turret** using **PID**

5 Shoot the **artifacts**

6 Drive to the next **intake** position using **odometry**

REPEAT



Autonomous Improvements

Our autonomous routines evolved a lot over the season. We started with a simple 9-ball path and expanded it into multiple high-scoring routines by adapting to rule changes, improving timing, and fixing what didn't work.

Adapting to the Gate Rule Change

When the gate rule changed, our original auton paths broke. We had to:

- change our starting angle
- adjust intake timing
- avoid the gate as it opened
- reposition our first cycle
- add new paths for the balls that fall into the HP zone

This forced us to redesign almost every auton we had.

What Worked

- odometry-based shooting positions
- auto-velocity for fast, accurate shots
- splitting auton into smaller actions
- parallel actions to save time
- transfer wheel logic to prevent jams
- turret PID for stable aiming

These made our auton much more reliable.

Final Result

By the end of the season, our auton was:

- faster
- more consistent
- safer with partners
- able to score more balls
- able to adapt to different match situations

These software improvements were some of the biggest reasons our robot performed well.

Building on the Original 9-Ball Path

Our first working auton was a 9-ball routine. We used it as the foundation for all later versions.

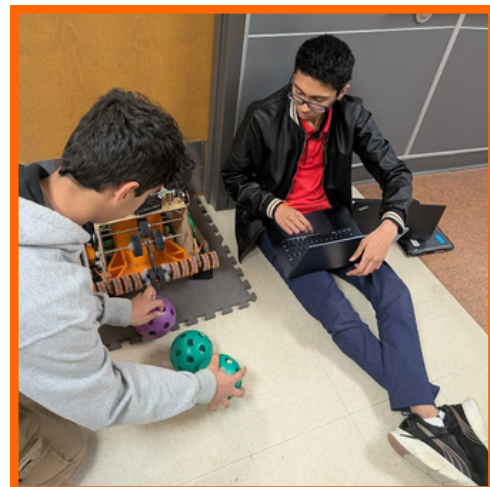
We expanded it by:

- adding gate-drop balls
- adding HP-zone sweeps
- improving timing between cycles
- switching to parallel actions (drive + intake + spin-up at the same time)
- using auto-velocity instead of fixed presets

Every improvement stacked on top of the original structure.

What Didn't Work (At First)

- trying to score too many balls too fast
 - using bang-bang for flywheel control
 - running everything in one giant action
 - ignoring gate timing
 - assuming the field coordinate system was "normal"
 - not accounting for partner autons
- We fixed each issue through testing and iteration.





Velocity PID

MAJOR PROBLEM/MAIN TOPIC

Tuning the velocity PID was one of the major problems during the autonomous process, requiring precise focus and attention be able to find the exact values which would result to consistent auton paths. This page dives deeper in one of the focal components of Velocity PID, explaining the needed outcomes, etc. for it.

WHAT IS VELOCITY PID?

A PID (Proportional-Integral-Derivative) controller works by calculating the difference, or error, between a desired value (the velocity setpoint) and a measured value (the actual motor velocity). It then applies a corrective motor power based on three adjustable coefficients: k_P (proportional), k_I (integral), and k_D (derivative).

We used FTC Dashboard and RoadRunner to evaluate values.

NORMAL PID

Normal (Positional) PID calculates output based on total error (Setpoint - Process variable). It usually winds up Integral when saturated which causes output bumps. It usually has better immunity to process (PV) noise as gains act on error (so no noisy changes).

KEY DIFFERENCES

1

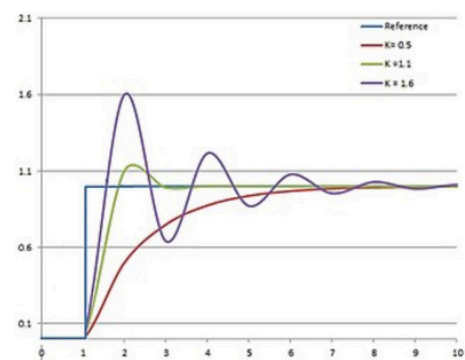
Output Basis: Position uses total error, Velocity uses change in error.

2

Tuning: Velocity allows bump-free tuning in auto-mode.

3

Implementation: Velocity form is derived from the positional form (by differentiation) and is often the default.



This image shows how Proportional-Integral-Derivative work together.



Velocity PID Tuning & Challenges

Why Velocity PID Was Hard to Tune

Tuning the flywheel's velocity PID was one of the most time-consuming parts of our autonomous development. Getting the flywheel to hit a single RPM target is simple, but getting it to recover instantly and repeat the same velocity three times in a row is much harder. Even small differences in velocity affect shot accuracy, so the tuning had to be extremely precise.

Multi-Shot Consistency

- Getting one artifact in consistently is easy, the flywheel has time to stabilize.
- Getting three artifacts in a row requires the flywheel to:
 - recover RPM instantly after each shot
 - avoid overshoot on the first shot
 - avoid undershoot on the second and third
 - stay stable even when the intake is running

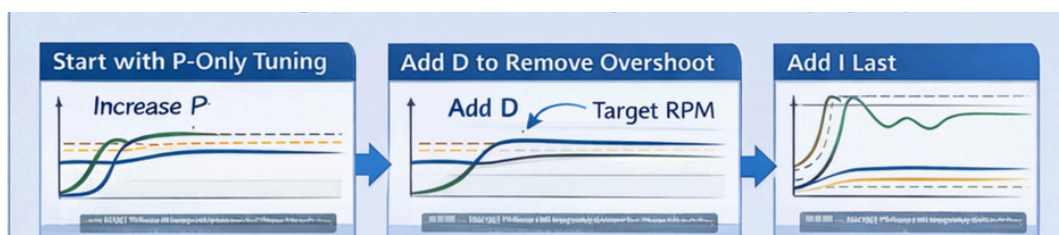
This is where Velocity PID mattered most.

What Made Tuning Difficult

- battery voltage changes affected RPM
 - intake current spikes caused small dips
 - PID values that worked for one shot didn't work for three
 - overshoot on the first shot caused inconsistent scoring
 - under-recovery after the second shot caused the third to fall short
- We had to tune for real match conditions, not ideal lab conditions.

How We Tuned Velocity PID

We used FTC Dashboard graphs to visualize RPM and adjust PID values step-by-step:
 start with P-only tuning Increase P until the flywheel reaches the target quickly without oscillating.
 add D to remove overshoot D smooths the curve and prevents the first shot from being too strong.
 add I last Only enough to correct long-term drift from battery sag or friction changes.
 test with full subsystem load We tuned with the intake, turret, and drivetrain running because voltage dips from other motors affect RPM.
 validate with 3-shot cycles We didn't finalize tuning until the flywheel could fire three consecutive shots with minimal RPM drop.



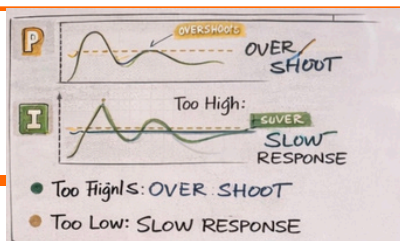


PID-Stabilized Flywheel And Turret

What is it?

PID control is one of the most important parts of our autonomous system. We use PID on both the flywheel and the turret to keep them stable, accurate, and consistent. PID helps the robot reach a target value (like an angle or RPM) and stay there without bouncing or drifting.

What Is PID?



PID stands for Proportional-Integral-Derivative. It's a control method that adjusts motor power based on how far the robot is from the target and how that error is changing.

PID helps the robot:

- accelerate smoothly
- reach a target setpoint
- hold that value steady
- avoid overshooting
- stay consistent even when battery voltage drops

This is critical for autonomous because every shot and every rotation must be stable.

How PID Helps the Turret

- smooth rotation
- no jerky movement
- no overshoot past the target
- holds angle even while the robot moves
- stays stable against defense bumps

This makes turret alignment extremely reliable.

How PID Helps the Flywheel

- P spins up the flywheel quickly
- I fixes slow drift
- D prevents overshoot

Without PID, the RPM would bounce up and down, causing inconsistent shots.

With PID, the flywheel reaches the target RPM and stays flat.

P — Proportional Term

The P term looks at the current error (how far we are from the target).

- Big error → big correction
- Small error → small correction

For the flywheel, P helps it spin up quickly. For the turret, P helps it rotate toward the target smoothly.

If P is too high, the system overshoots. If P is too low, it feels slow or sluggish.

I — Integral Term

The I term looks at past error and fixes long-term drift.

It helps when:

- the flywheel slowly falls below the target RPM
- the turret is slightly off because of friction or weight
- battery voltage drops during a match

I "pushes" the system back to the exact target over time.

Too much I causes wobbling or slow oscillations, so we keep it small.

D — Derivative Term

The D term looks at how fast the error is changing.

It acts like a shock absorber:

- prevents overshoot
- smooths out sudden movements
- stops the turret from swinging past the target
- keeps the flywheel from spiking above the RPM

D makes the system feel controlled and stable.



Roadrunner Tuning

Main Idea

Road Runner is the backbone of our autonomous movement, but it only works well if the drivetrain is tuned correctly. Small errors in track width, wheel radius, or acceleration constraints can cause the robot to drift, overshoot, or miss its scoring positions. This page explains how we tuned Road Runner to make our auton paths consistent across different fields.

Drive Geometry Calibration

We started by calibrating the robot's physical constants so Road Runner's internal model matched real-world movement.

- Track width calibration We ran repeated in-place turns and adjusted the track width until the robot rotated exactly 360° with minimal error.
- Wheel radius calibration We drove a long straight path and tuned the wheel radius so the measured distance matched the actual distance.
- Lateral multiplier tuning Strafing introduces natural drift, so we tuned the lateral multiplier until sideways movement matched Road Runner's predicted displacement.

These calibrations ensured that the robot's pose estimate stayed accurate throughout long auton cycles.

Pinpoint + Road Runner Integration

Pinpoint odometry gave us a more accurate pose estimate than wheel odometry alone. We used it to:

- correct Road Runner's pose at key points
- reduce drift over long paths
- improve turret aiming math by providing cleaner X/Y/ θ data

This fusion made our auton paths repeatable across different tournaments.

Validation Process

We validated tuning using a structured testing routine:

- 10-run straight-line test
- 10-run strafe test
- multi-turn heading test
- full-cycle auton test with intake + turret + flywheel running
- cross-field drift test to check long-distance accuracy

We only finalized tuning once the robot consistently ended within a few centimeters of the target across all tests.

Heading Drift Correction

- Even with perfect geometry, mecanum drivetrains naturally drift. We corrected this by:
- tuning the heading PID inside Road Runner
- validating IMU stability across multiple matches
- running long-distance tests to check for cumulative drift
- Stable heading was critical for turret alignment and shot accuracy.



GoBilda Pinpoint

What Pinpoint Actually Does

Our autonomous system is built around the GoBilda Pinpoint Odometry Computer, which gives us real-time X/Y position and heading on the field. Pinpoint combines:

- **three dead wheels** (tracking lateral + forward movement)
- **an onboard IMU** (tracking rotation and correcting drift)

This fusion gives us a pose estimate that is far more stable than using either method alone.



Pinpoint Odometry Controller

Why Pinpoint Improves Pose Estimation

Pinpoint's strength comes from how it blends IMU and odometry data:

- Odometry tracks exact linear movement but drifts when wheels slip.
- IMU tracks rotation accurately but drifts over long periods.
- Pinpoint fuses both, correcting each system's weaknesses.

This gives us:

- lower drift over long paths
- more accurate heading during turns
- consistent position tracking even on worn or uneven fields
- repeatable autonomous paths across different tournaments

This is why our autonomous routines behave the same way every run.

Two-Wheel vs. Three-Wheel Odometry

Pinpoint supports both, but we use three-wheel because it gives the most reliable pose estimate.

Two-Wheel Odometry

- Tracks forward/backward movement
- Tracks rotation
- Cannot directly measure sideways drift
- Needs IMU fusion to estimate lateral movement

Three-Wheel Odometry

- Tracks forward movement
- Tracks sideways movement
- Tracks rotation
- Directly measures all motion components
- More accurate for strafing robots (like mecanum)

Because our drivetrain strafes frequently, three-wheel odometry gives us cleaner, more stable pose data.

How We Use Pinpoint in Autonomous

Pinpoint gives us the robot's live position (x, y, θ) . We use that to calculate:

- distance to the target
- turret rotation angle
- hood angle
- flywheel velocity

This turns aiming into a geometry problem, not a vision problem.

Example

$$d = (x_t - x_r)^2 + (y_t - y_r)^2$$

$$\theta = \text{atan2}(y_t - y_r, x_t - x_r)$$

Pinpoint provides the robot's pose, and we compute the rest.



Odometry

What is it?

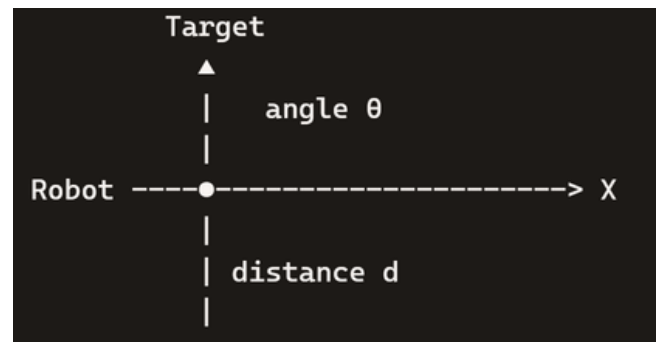
The robot uses odometry to compute the distance and angle to the target.
This gives us consistent aiming without needing a camera during autonomous.

Robot and Target Positions

- Robot odometry position: (x_r, y_r)
- Target field position: (x_t, y_t)

Distance Calculation

$$d = \sqrt{(x_t - x_r)^2 + (y_t - y_r)^2}$$



Turret Angle Calculation

$$\theta = \text{atan2}(y_t - y_r, x_t - x_r)$$

This graph shows how the distance is calculated through PID from the starting point

HOOD ANGLE/FLYWHEEL VELOCITY

We use distance from the goal to calculate:

- hood angle
- flywheel velocity

WHAT IS ODOMETRY?

Odometry is a form of localization that uses data from sensors like encoders to derive an estimated position relative to a starting point. Localization is a means for being able to locate the position of the bot at some point in time. Odometry is especially useful in autonomous programs because it allows for easier implementation of different tasks on the field due to understanding one's position.

Pose



AprilTag/Vision

WHAT IS AN APRIL TAG?

An AprilTag is a type of black-and-white visual marker, similar to a QR code, used in robotics and computer vision for **localization, navigation, and object identification**. By detecting an AprilTag with a camera, a computer system can quickly determine the tag's **exact position and orientation** in 3D space relative to the camera, providing **precise, real-time data** for tasks like guiding robots to specific locations or calibrating cameras.

HOW IT WORKS

1. Detection: A camera captures an image and uses **specialized software** to find the AprilTag pattern within the image.
2. Identification: The unique pattern of the AprilTag identifies its specific tag.
3. Pose Estimation: By analyzing the detected tag, the software **calculates its precise position (x, y, z coordinates) and orientation (roll, pitch, yaw)** relative to the camera.

STEPS FOLLOWED

1. We began by building foundational knowledge and setting up the basic code structure. This initial phase is critical for understanding the tools we're working with.

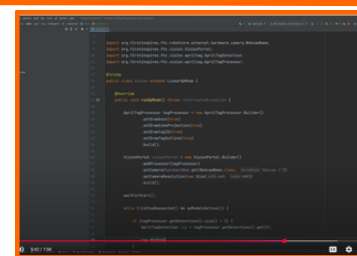
2. We wrote code to get the webcam and vision sensor communicating and performing the most basic task: identifying the AprilTag.

3. We achieved a crucial milestone—the AprilTag was detected! This validated our basic setup and allowed us to move on to the next, more complex task.

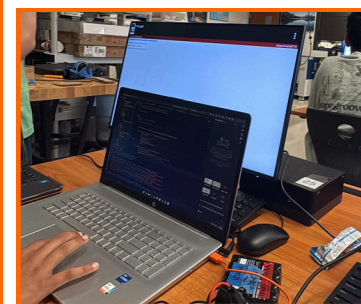
4. We hit a new roadblock when trying to get the pose data, receiving a "null" error. This is a great example of how getting one thing right often exposes a new layer of complexity. We correctly identified that this error is often related to incorrect configuration or physical setup.

5. We discovered the root cause was an incorrect AprilTag library setup. This is a powerful lesson: sometimes the problem isn't in your logic, but in an external dependency or a foundational configuration detail. By adding `AprilTagLibrary aprilTagLibrary = AprilTagGameDatabase.getCurrentGameTagLibrary();`, we correctly initialized the library for our specific application.

6. With the library correctly configured, our code was able to successfully retrieve the pose data (X, Y, Z, Yaw, Pitch). We then went a step further by testing different positions to fully understand the meaning of each data point, which is a fantastic way to validate and gain confidence in our solution.



The code



Team working



Vision camera detecting



AprilTag Goal Alignment

Goal Alignment

AprilTags are mainly used to help the robot figure out exactly where the goal is and how utterly should aim. When the camera sees an AprilTag placed near the target, it can calculate the robot's position and target. That information is what lets the robot line up accurate shots.

PRIMARY ALIGNMENT JOB:

PRECISE ANGLE TO GOAL

The tag gives the robot the yaw angle between its camera and the target, so the turret knows which direction to rotate.

DISTANCE TO THE GOAL

Using the tag's size and position in the camera image, the robot can estimate how far away it is. This helps set flywheel speed and hood angle.

ROBOT POSE RELATIVE TO THE FIELD

AprilTags give a full 6-DOF pose (x, y, z, yaw, pitch, roll), which helps the robot understand where it is even if odometry drifts.

AUTOMATIC ALIGNMENT CORRECTIONS

If the robot is slightly off-center, the AprilTag data tells the turret how much to rotate to line up perfectly with the goal.

RELIABLE AIMING EVEN WHEN SHOOTING

Because the tag updates quickly, the robot can adjust its aim in real time instead of relying on static math.

In short: AprilTags give the robot the exact angle and distance it needs to aim at the goal, which is why they are so useful for alignment systems



AprilTag AutoAlign

Key Components

How PID corrects for the bearing

The PID controller calculates a turnPosition based on the AprilTag's bearing to direct the robot's corrective turn.

- **Positive bearing:** A positive bearing indicates the AprilTag is to the right of the camera's center. The PID output will be a positive value, telling the robot to turn clockwise. Our code sets the motors to turn the robot to the right: `frontLeft.setTargetPosition(-turnPosition)`, `frontRight.setTargetPosition(turnPosition)`, and so on.
- **Negative bearing:** A negative bearing means the tag is to the left. The PID output will be negative, and the robot will turn counter-clockwise.
- **Zero bearing:** When the robot is facing the tag directly, the bearing is zero. The PID output will also be zero, and the robot's turning will cease.

WHAT DOES IT USE?

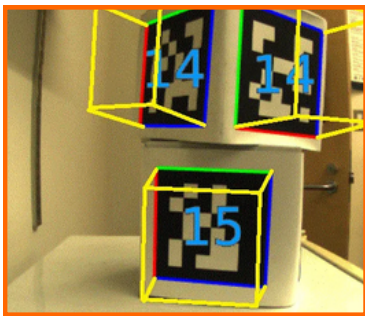
It uses pre-relative position places that is delivered into the webcam though the pose data which signals it to move the **autonomously align to match** and face the AprilTag.

WHY IS IT IMPORTANT?

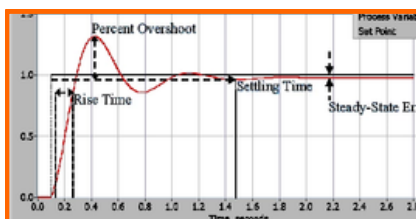
AutoAlign implementation is crucial to the game because it **stabilizes the consistency of the apriltag detection**. That simply just optimizes the functionality of the bot by **reducing driver time for positioning** and **scoring the artifacts in the right goal**.

WHY DO WE USE PID?

We use PID for automatic control and consistent stability which reassures the values inputted in order for it to reach a desired setpoint and endpoint.

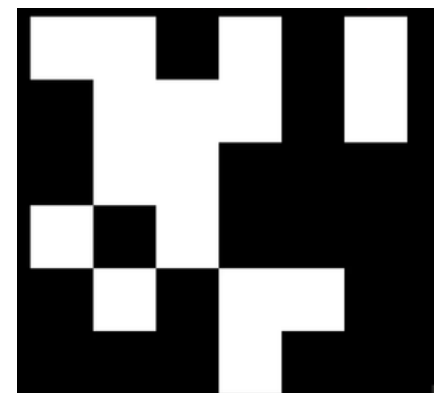


This picture shows an example of an AprilTag identification through a webcam.

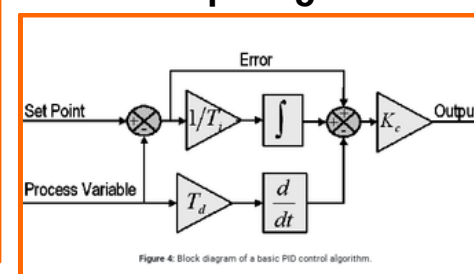


This shows the tuning process of PID in a graph representation.

There are two main components to AutoAlign, **AprilTag controller PID, and vision processing to identify and perform its job**. The AprilTag controller enables the use of PID in it, **calculated by calculating the bearing, to correct it and set it to its right position**. Vision processing is the main code that **allows the webcam to turn on** to enable the main way for it to identify it.



Apriltag



PID error through a flowchart



Limelight Vision System

The Limelight is our robot's primary vision sensor for detecting AprilTags and aligning to field targets. While our autonomous aiming is mostly math-based, Limelight gives us an additional layer of correction by providing real-time visual feedback about the robot's position and angle relative to the goal.



What Limelight Actually Does for Us

Limelight processes AprilTags on the field and returns:

- robot-to-tag translation (how far we are from the target)
- robot-to-tag rotation (the yaw angle we need to correct)
- pose estimation using AprilTag geometry
- latency-compensated measurements for smoother alignment

This information helps the robot understand exactly where the goal is, even if odometry drifts slightly.

Limelight in TeleOp

During TeleOp, we call our vision alignment function inside the update loop. Limelight continuously adjusts:

- drivetrain heading
- turret rotation
- distance-based shot presets

This gives the drivers a "snap-to-target" feel, the robot naturally lines itself up so the driver can focus on cycling instead of aiming.

Limelight in Autonomous

Even though most of our auton aiming is math-driven, Limelight helps by:

- verifying turret angle
- correcting small heading errors
- providing backup pose data if odometry drifts
- helping stabilize long-distance shots

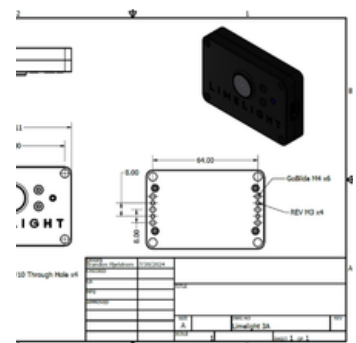
This makes our auton more robust on fields with inconsistent carpet or friction.

Why Limelight Matters

Limelight isn't just a camera, it's a full vision system that improves:

- shot accuracy
- alignment speed
- auton reliability
- TeleOp consistency
- pose estimation when combined with odometry

It acts as a second source of truth for the robot's position, making the entire scoring system more stable.



This shows the inner structure/components of limelight



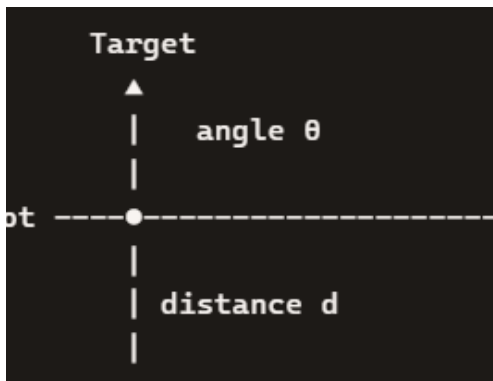
Diagrams

Diagram – Flywheel PID Behavior



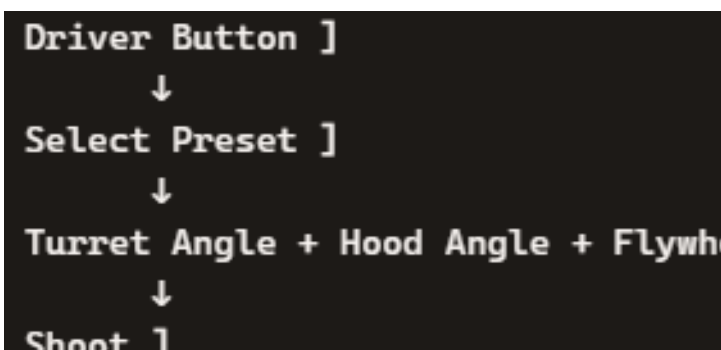
The flywheel accelerates **smoothly** up to the **target** RPM without **overshooting** and then maintains a **steady**, flat **speed**. The PID controller **continuously** adjusts motor output to **correct** small **disturbances**, keeping the RPM **stable** even when **battery** voltage **drops** or the load on the flywheel changes.

Diagram – Turret Aiming



We use the robot's **odometry** to determine its **position** on the field and **calculate** the **distance d** and **angle θ** to the target. The turret automatically rotates to the required **angle θ** , while the **hood position** and **flywheel speed** are adjusted based on the calculated distance d to ensure **accurate** shots.

Diagram - Preset Buttons



A single **button** press **activates** a **preset** that automatically sets the **turret** angle, **hood** angle, and **flywheel RPM**, simplifying operation and improving **consistency**.